TRIRACH Adam 1G2

Fichier explicatif du Projet 7 : Mine Game Hub

Ce projet a été réalisé dans le cadre de la NSI. Il s'agit d'un site web de mini-jeux intelligents, développé en Python avec le micro-framework Flask, et intégrant une interface web HTML/CSS/JavaScript.

Ce projet a permis de réinvestir quatre notions vues dans le programme de NSI première : programmation en Python, HTML, CSS et JavaScript.

Le site propose quatre jeux interactifs :

- Le Pendu
- Le Quiz des Drapeaux
- Le Memory
- Devine le Nombre

Chaque jeu possède sa propre page et ses propres mécanismes, tout en s'intégrant dans une interface commune. Les informations concernant les commandes utilisées sont ici et dans le code en commentaire.

I – Lancer le site

Le site ne fonctionne pas comme une simple page HTML : est propulsé par Flask. Pour faire fonctionner le site en local, il faut donc suivre les étapes suivantes :

Prérequis : avoir Python installé (version 3.7 ou plus) **et a**voir Flask installé (via « pip install flask »). L'utilisation de Visual Studio Code est très recommandée.

L'**organisation des fichiers** dans le dossier « Projet_7_Adam_TRIRACH » est la suivante :

- Le fichier source « app.py ».
- Le dossier « static » contenant les images, les drapeaux en format svg, les fichiers JavaScript, les données concernant les drapeaux (« countries.json ») et le fichier style CSS.
- Le dossier « data » avec dedans un fichier texte comprenant les mots à deviner pour le pendu.
- Le dossier « templates » contenant les fichiers html.

```
├── app.py
├── templates/
│   ├── index.html
│   ├── pendu.html
│   ├── drapeaux.html
│   ├── memory.html
│   └── nombre.html
├── static/
│   ├── style.css
│   ├── js/
│   │   ├── drapeaux.js
│   │   ├── memory.js
│   │   └── nombre.js
│   ├── svg/          (drapeaux)
│   ├── img/          (images des jeux)
│   └── countries.json
└── data/
    └── mots_pendu.txt
```

Lancement du serveur Flask :

1. Ouvrir un terminal dans le dossier du projet
2. Exécuter la commande « python app.py »
3. Le terminal affiche alors quelque chose comme : * Running on http://127.0.0.1:5000/
4. Ouvrir un navigateur web et se rendre à l'adresse : http://127.0.0.1:5000/

Le site est maintenant accessible localement, et tous les jeux peuvent être lancés depuis la page d'accueil.

II – Fonctionnement de Flask et de la page d'accueil

Le projet utilise **Flask**, un micro-framework Python permettant de créer des applications web.

Voici les éléments clés du fichier app.py :

- app = Flask(__name__) : Crée l'application Flask.
- app.secret_key = "..." : Nécessaire pour gérer les sessions (utilisé pour le jeu du pendu).

Chaque page du site (y compris les jeux) est gérée par une route Flask :

```
@app.route("/")
def index():
    return render_template("index.html")
```

Cela signifie que quand l'utilisateur accède à l'adresse racine (/), Flask renvoie le fichier index.html qui se trouve dans le dossier templates/.

La page d'accueil (index.html) contient :

- Un titre et un slogan
- Une **grille 2x2** de jeux, chacun sous forme de carte cliquable avec une image et un titre
- Un **pied de page** avec le nom du développeur et la mention du projet NSI

Le serveur est démarré par :

```
if __name__ == "__main__":
    app.run(debug=True)
```

Le mode debug=True permet d'afficher les erreurs et de recharger le serveur automatiquement pendant le développement.

III – Le Pendu

A. Déroulement du jeu (côté joueur)

L'utilisateur doit deviner un mot mystère choisi aléatoirement parmi une liste de 140 mots. À chaque tour, il propose une lettre : si la lettre est dans le mot, elle est révélée ; sinon une erreur est comptabilisée. Le joueur a droit à 10 erreurs maximum. Quand il gagne ou perd, un bouton Rejouer s'affiche.

B. Fonctionnement du code

Démarrage d'une nouvelle partie :

« if "mot" not in session: » : si la session (données temporaires mémorisées pour un utilisateur) ne contient pas encore de mot à deviner : on pioche un mot dans data/mots_pendu.txt. On initialise la session :

- session"mot" => mot choisi
- session"trouvees" => lettres trouvées
- session"erreurs" => compteur d'erreurs

- session"lettres_proposees" => historique des lettres essayées

« **session »** est un dictionnaire temporaire propre à chaque utilisateur. Il permet de **garder en mémoire l'état du jeu**.

Traitement d'une lettre envoyée :

Quand le joueur valide une lettre (méthode POST) :

lettre = request.form"lettre".lower() : request.form... récupère la lettre envoyée par le formulaire et .lower() transforme en minuscule.

Ensuite, si la lettre est non valide ou déjà proposée => message d'erreur ; sinon :

- Si elle est dans le mot => on l'ajoute à trouvees
- Sinon => erreurs += 1

Affichage du mot

Le mot est reconstruit avec des tirets pour les lettres non encore trouvées.

Fin du jeu

On vérifie si toutes les lettres ont été trouvées. Puis on renvoie vers le fichier HTML pendu.html en lui transmettant les données actuelles (mot à afficher, erreurs, message, etc.) : return render_template("pendu.html", mot_affiche=..., erreurs=..., ...).

C. La page HTML

Les {{ ... }} permettent d'afficher les variables Python envoyées par render_template.

{% ... %} est une instruction conditionnelle : ici, on n'affiche le bouton "Rejouer" que si la partie est finie.

IV – Le Quizz des Drapeaux

A. Déroulement du jeu

Dans ce jeu, l'utilisateur a 2 minutes pour retrouver 10 pays à partir de leur drapeau.
Les drapeaux sont affichés dès le début et il faut simplement taper le nom d'un pays dans le champ prévu. Dès qu'un nom correct est saisi, le drapeau concerné passe en fond vert clair avec une bordure verte, et le champ est vidé automatiquement. Il n'est pas nécessaire d'appuyer sur entrée, le jeu reconnaît la saisie en direct.

Le joueur peut rejouer ou revenir à l'accueil à tout moment grâce aux liens affichés sous la grille.

B. Détails du code

Fonctionnement technique

La page HTML contient un champ texte <input id="reponse"> où l'utilisateur tape sa réponse. Les drapeaux sont affichés dans et mis à jour via le fichier JavaScript drapeaux.js.

Le JavaScript s'exécute uniquement quand le DOM est prêt grâce à document.addEventListener('DOMContentLoaded', ...).

Gestion du temps

Un timer est affiché dans la balise et décrémenté toutes les secondes avec setInterval(...).
La fonction clearInterval(timerId) est utilisée pour stopper le compte à rebours à la fin.

Normalisation des réponses

Pour que l'utilisateur ne soit pas pénalisé par les accents, majuscules, ou espaces, la saisie est transformée par la fonction normaliserText() qui utilise :

- str.normalize('NFD') pour séparer les accents
- str.replace(/\u0300-\u036f/g, '') pour retirer les diacritiques
- str.toLowerCase() pour convertir en minuscules
- str.replace(/\s\-\'/g, '') pour supprimer les espaces, tirets, apostrophes…

Cela permet, par exemple, de reconnaître "Etats Unis", "États-Unis" ou "etatsunis" comme une même réponse.

Fichier countries.json

Le fichier countries.json contient les noms de **tous les pays** du monde au format JSON. Chaque pays y est représenté par trois données :

- "name" : nom du pays en français (ex : "France") (les noms étaient initialement en anglais mais ont été traduit manuellement par moi-même).
- "alpha-2" : code officiel à deux lettres ISO 3166-1 (ex : "FR" pour France)
- "country-code" : code numérique (non utilisé ici)

Le code alpha-2 est utilisé pour faire le lien entre les noms des pays et les fichiers SVG des drapeaux.

Fichiers SVG (Scalable Vector Graphics)

Chaque drapeau est un fichier svg placé dans le dossier static/svg.
Le nom du fichier est le code alpha-2 du pays en minuscules (ex : fr.svg, us.svg, jp.svg).

Le format SVG est idéal pour les drapeaux car :

- Il est vectoriel (pas de perte de qualité en zoomant)
- Léger à charger (moins de 4 Mo pour l'ensemble de 200 pays
- Facile à styliser avec CSS ou JavaScript

Les fichiers ont été récupérés sur Github (https://github.com/hampusborgos/country-flags/tree/main/svg) mais certains drapeaux ont été supprimés manuellement (ainsi que leurs informations dans le fichier json) car non considéré comme Etats à part entière (ex : Iles Cook).

Sélection aléatoire des 10 drapeaux

La fonction randomFlags(...) utilise l'algorithme de Fisher-Yates pour mélanger aléatoirement les pays du tableau data.
C'est un algorithme efficace et sans biais qui fonctionne de la façon suivante :

- On parcourt le tableau à l'envers
- À chaque étape, on échange l'élément courant avec un élément aléatoire parmi ceux précédents
- On coupe ensuite le tableau avec slice(0, n) pour ne garder que les n premiers (ici 10)

Reconnaissance des bonnes réponses

À chaque nouvelle saisie, le script :

- Transforme la saisie avec normaliserText
- Vérifie si le texte correspond à une des 10 réponses attendues
- Si oui : Il ajoute le pays aux réponses devinées avec guessed.add(...) (évite les doublons) ; change le style des drapeaux concernés avec querySelectorAll(...) + style.backgroundColor et style.border ; vide automatiquement le champ de saisie input.value = ''.

Quand tous les pays sont trouvés (c'est-à-dire guessed.size === paysSelect.length), le message de victoire s'affiche avec messageFin.textContent = "..." et le jeu se bloque.

V – Memory

A. Déroulement du jeu

Le jeu Memory repose sur le principe classique : il faut retrouver des paires identiques parmi des cartes retournées. Ici, les cartes représentent des objets liés à l'informatique : processeur, souris, clavier, réseau, etc. L'objectif est de retourner deux cartes identiques d'affilée pour qu'elles restent visibles. Si elles sont différentes, elles se retournent à nouveau après un court délai. Une partie est gagnée lorsque toutes les cartes (soit 10 paires) ont été trouvées.

Au chargement de la page, 20 cartes (soit 10 paires) sont placées de manière aléatoire dans une grille. Lorsque l'utilisateur clique sur une carte celle-ci se retourne. Si une seconde carte est retournée et qu'elle correspond à la première, les deux restent visibles. Sinon, elles sont cachées à nouveau après une pause de 800 millisecondes. Un message de félicitations s'affiche automatiquement lorsque toutes les cartes ont été découvertes. L'utilisateur peut rejouer en cliquant sur le bouton Rejouer.

B. Détails du code

Fonctionnement technique

Le plateau de jeu est représenté par .
Les images des cartes sont stockées dans le dossier static/img/cartes et nommées selon ce qu'elles représentent.

Dès que le DOM est prêt, la fonction initialiserJeu() est appelée grâce à document.addEventListener("DOMContentLoaded", ...). Cette fonction :

- Vide le plateau avec plateau.innerHTML = ""
- Duplique le tableau des images pour avoir deux exemplaires de chaque élément avec cartes = ...imagesCartes, ...imagesCartes
- Mélange les cartes avec la fonction melanger()
- Crée dynamiquement les éléments HTML pour chaque carte avec document.createElement("div") et appendChild(...)

Mélange aléatoire

Le mélange des cartes utilise l'algorithme de Fisher-Yates, le même que dans le jeu des drapeaux.

Interaction et logique de jeu

Lorsqu'une carte est cliquée, la fonction retournerCarte(...) est appelée.

Cette fonction :

- Vérifie si la carte est déjà retournée ou si le jeu est temporairement verrouillé (en cours d'animation) avec classList.contains("visible")
- Ajoute la classe visible à la carte pour la retourner visuellement
- Si une première carte avait déjà été retournée : compare les deux cartes en utilisant dataset.nom ; si elles correspondent, elles restent visibles ; sinon, elles sont cachées après un délai de 800 ms avec setTimeout(...).

Le verrouillage temporaire des clics pendant l'animation est géré grâce à une variable booléenne « verrouillage ».

Quand toutes les cartes ont été trouvées, la victoire est détectée grâce à document.querySelectorAll('.carte.visible').length === 20, et le message Bravo ! Tu as gagné ! est affiché dans <p id="etat">.

VI – Devine le nombre

A. Déroulement du jeu

Ce jeu propose à l'utilisateur de retrouver un nombre secret compris entre 1 et 10 000, en moins de 60 secondes. Le joueur entre un nombre dans un champ prévu, et le jeu lui indique si le nombre mystère est plus grand ou plus petit. La partie se termine soit quand l'utilisateur trouve le bon nombre, soit quand le temps est écoulé. Un bouton permet de recommencer une partie à tout moment.

À l'ouverture de la page, seule l'interface est visible mais bloquée : il faut d'abord cliquer sur le bouton Commencer pour lancer une partie. Le chronomètre se lance, un nombre mystère est généré aléatoirement, et le champ de saisie devient fonctionnel. Chaque fois qu'un nombre est validé, le champ est vidé automatiquement et le jeu affiche un message d'indice dans <p id="indice"> : soit "c'est plus", soit "c'est moins". Si le nombre est correct, la partie est gagnée. Sinon, le joueur continue jusqu'à trouver ou jusqu'à la fin du timer.

B. Détails du code

Génération du nombre mystère

Le nombre mystère est généré avec Math.floor(Math.random()*10000) + 1. La fonction Math.random() retourne un nombre entre 0 et 1, multiplié par 10000 et arrondi par défault à l'entier avec Math.floor(...). Le +1 permet de s'assurer que le nombre sera compris entre 1 et 10 000 inclu.

Gestion du chrono

Le chronomètre est géré à l'aide de setInterval(..., 1000), ce qui exécute une fonction toutes les 1000 millisecondes, donc chaque seconde. À chaque intervalle, tempsRestant—se décrémente, puis le format d'affichage est mis à jour avec grâce à ${minutes}:${secondes.toString().padStart(2, '0')}. Lorsque le temps arrive à zéro, la fonction terminerJeu(false) est appelée et la partie est bloquée.

Validation d'un nombre

L'utilisateur peut valider un nombre de deux façons : en cliquant sur le bouton Valider ou en appuyant sur la touche Entrée. La saisie est récupérée avec parseInt(...) et vidée avec champ.value = "". Si la saisie n'est pas un nombre, le message Ce n'est pas un nombre valide est affiché. Sinon, on compare la saisie au nombre mystère :

- Si la saisie est inférieure => le jeu affiche « C'est plus ! »
- Si elle est supérieure => le jeu affiche « C'est moins ! »
- Si elle est exacte => la fonction terminerJeu(true) est appelée.

Fin de partie

Que l'utilisateur gagne ou perde, la fonction terminerJeu(...) désactive le champ et le bouton avec disabled = true, arrête le timer avec clearInterval(timer), et affiche un message final dans <p id="message-fin">. S'il a gagné, le message est « Bravo ! Tu as trouvé le nombre mystère : ». Sinon, le jeu révèle le nombre à l'aide du message « Temps écoulé ! Le nombre était : ».

VII – Style du site (CSS)

Apparence générale

La règle body définit la police utilisée (font-family: Arial, sans-serif), le centrage du texte (text-align: center) et le fond d'écran avec un dégradé (background: linear-gradient(135deg, #d6eaff, #f1f8ff)). Les titres h1, h3, h4 ont tous la couleur color: #333.

Boutons

Les boutons sont stylisés avec font-size, padding, border-radius et background-color: #1e90ff. Au survol, l'effet button:hover utilise transform: scale(1.05) ou un changement de couleur (#155fa0) pour indiquer que le bouton est interactif.

Jeu des drapeaux

La grille de drapeaux est gérée par l'élément #drapeaux avec display: flex, flex-wrap: wrap et gap: 15px pour les espacements. Chaque drapeau utilise la classe .drapeau avec width: 120px, un fond blanc, une bordure grise claire, et une transition fluide avec transition: all

0.3s ease. Le champ de saisie #reponse est agrandi avec font-size: 18px et width: 250px.
Le chrono s'affiche en rouge grâce à #timer { color: #d63333 }.

Jeu memory

La grille du memory est définie avec #plateau : une grille de 5 colonnes (grid-template-columns: repeat(5, 100px)) et des espaces avec grid-gap: 10px. Les cartes ont une taille fixe de 100px × 100px avec border-radius: 8px, une couleur grise claire, et un affichage centré via display: flex, justify-content: center et align-items: center. L'image de chaque carte est cachée par défaut (.carte img { display: none }) et n’apparaît que quand la carte a la classe .visible, grâce à .carte.visible img { display: block}.

Accueil : sélection des jeux

Les jeux sont présentés sur la page d’accueil sous forme de cartes dans une grille de 2 colonnes avec grid-template-columns: repeat(2, 1fr) dans .grille-jeux.
Chaque carte de jeu .carte-jeu a un fond blanc, une ombre portée avec box-shadow, un coin arrondi (border-radius: 12px), et un effet d’agrandissement au survol (transform: scale(1.05)).
L’image de chaque jeu est affichée à 100px × 100px, centrée, avec object-fit: contain.

Pied de page

Le pied de page footer est simple : fond gris clair, texte centré, et une bordure supérieure.